CSED233 · 데이터구조 · 포스텍

정리본 2. C++ 기초

자료구조 구현의 토대: 왜 C++인가 · 빌드 프로세스 · 포인터와 메모리 · new/delete.

1. 왜 자료구조를 C++로 배우는가

자료구조는 포인터 기반의 자료 조직이다. 따라서 메모리를 직접 제어할 수 있는 언어가 필요하다.

- **C (1972)**: 절차지향, 메모리 직접 관리, 함수 중심. 빠르지만 위험하다.
- **C++**: C + 객체지향 + 개선된 메모리 관리. 안전하면서 고성능 — 자료구조 학습에 "딱 맞는 추상화 수준".

1.1 다른 언어와 비교하면 이유가 보인다

Python — 구현은 쉽지만 본질을 숨긴다

```
arr = [0] * 1000000   # 내부 구조? 메모리는? 포인터는? 전부 모른다
class Node:
    def __init__(self, data):
        self.next = None   # Python이 포인터를 대신 관리
```

Python도 객체 참조는 있지만 주소 연산과 수동 메모리 관리를 추상화해 감춘다. 자료구조의 진정한 이해 = 메모리 레벨의 이해인데, Python에서는 그 원리를 직접 다루기 어렵다.

Java — 자동화가 원리 이해를 막는다

```
LinkedList<Integer> list = new LinkedList<>();
list.add(10);   // 내부 동작? 메모리 할당/해제 시점? GC의 영향?
```

세부사항이 자동화되어 있어 동작 원리와 성능 분석이 어렵다(Garbage Collection의 영향).

C++ — 메모리 관리가 프로그래머의 책임

```
int* arr = new int[n];  // 할당이 명시적
arr[i] = value;         // 접근이 명시적
```

```
delete[] arr;          // 해제가 명시적

struct Node {
    int data;
    Node* next;        // 포인터로 다음 노드 연결
};
```

자료구조가 메모리를 어떻게 쓰는지 명확히 보이고, 성능 분석이 가능하고, 포인터 개념을 완전히 이해하게 된다.

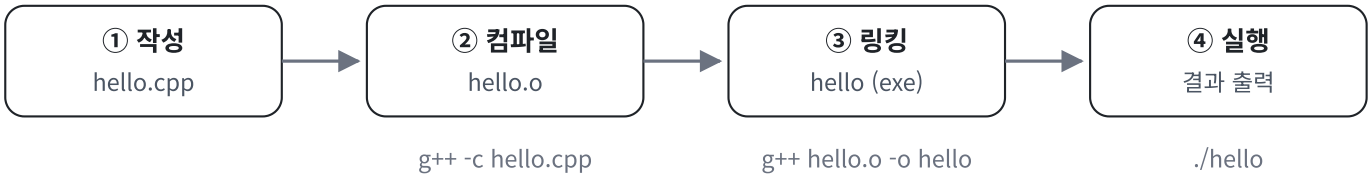
1.2 C++이 필수인 3가지 이유

1. **메모리 직접 관리** — 포인터로 노드를 연결하는 연결 리스트/트리/그래프, 크기를 제어하는 동적 배열. Python/Java의 가비지 컬렉션은 이 구조 이해를 어렵게 한다.
2. **성능 측정이 정확** — C++은 빨라서 O(n) vs O(log n) 차이가 명확하다 (n=100만: 100만 회 vs 20회). 느린 언어는 런타임 오버헤드가 커서 차이가 측정하기 어려워질 수 있다 (물론 실측은 보조 자료이고 복잡도는 점근 분석으로 판단).
3. **개념과 메모리가 연결** — "왜 배열은 Random Access가 O(1)인가"(연속 메모리 + 포인터 산술), "왜 연결 리스트는 Insert가 O(1)인가"(포인터만 변경) 같은 질문의 답이 C++에서 명시적으로 드러난다.

1.3 C++ 특징 정리

특징	설명	자료구조 학습에 중요한 이유
크로스플랫폼	Windows, Mac, Linux 모두 호환	어디서나 학습 환경 구성 가능
고성능	빠른 실행 속도	O(n) vs O(log n) 차이가 명확히 보임
메모리 제어	new/delete로 동적 할당	**가장 중요** — 포인터 기반 자료구조 구현의 핵심
객체지향	클래스로 자료구조 캡슐화	트리/그래프 같은 복잡 구조 관리 용이
최신 표준	C++11/14/17/20/23	모던 문법으로 간결한 코드

2. 빌드 프로세스: 작성에서 실행까지



각 단계는 종류가 다른 오류를 잡는다. 셋 모두 필요하다.

단계	하는 일	잡아내는 오류
② 컴파일 (Compilation)	C++ 코드를 기계어로 번역, 문법·타입 검사 → .o(object) 파일. 아직 라이브러리가 없어 실행 불가.	컴파일 오류 — 문법/타입, 선언 누락 (예: `#include "LinkedList.h"` 빼먹음)
③ 링킹(Linking)	.o 파일들 + 표준 라이브러리(iostream 등)를 결합 → 최종 실행 파일.	링커 오류 — 정의되지 않은 함수 (예: `undefined reference to 'sqrt'` — -lm 누락)
④ 실행 (Execution)	프로그램 실행, 결과 출력.	런타임 오류 — 실제 동작 중 발생

여러 파일 프로젝트

```
main.cpp ──[컴파일]──> main.o ┐
LinkedList.cpp ──[컴파일]──> LinkedList.o ├──[링킹]──> app ──[실행]──> 출력
                              stdlib ──┘
```

컴파일은 "각 파일이 맞는가?", 링킹은 "전부 모아 완성됐는가?", 실행은 "실제로 작동하는가?"를 본다.

3. Header 파일과 라이브러리

`#include <iostream>`은 "이런 함수·객체들이 있다"는 선언을 가져오는 것이다. 실제 구현은 컴파일러가 제공하는 라이브러리에 있다.

- `std::cout` — 표준 출력 객체: `cout << "Hello" << endl;`
- `std::cin` — 표준 입력 객체: `cin >> x;`

- `std::endl` — 줄바꿈 + 출력 버퍼 flush (줄바꿈만 필요하면 `'\n'`).

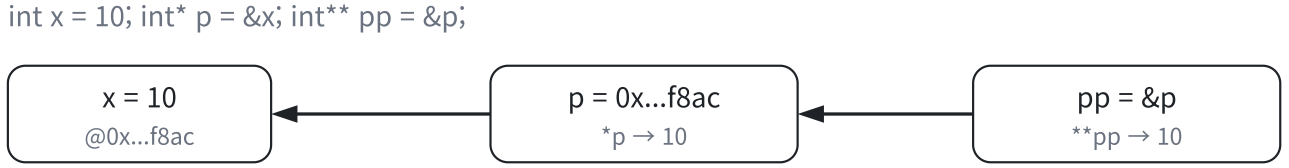
3.1 자료구조 학습에 필요한 헤더

Header	기능	자료구조 용도
`<iostream>`	입출력 (cout, cin)	데이터 입력/출력 테스트
`<cstdlib>`	표준 함수 (malloc, rand)	메모리 할당 (구형 — new가 나음)
`<cstring>`	문자열 함수 (strlen, strcpy)	문자열 자료구조
`<vector>`	동적 배열 (STL)	배열 자료구조 참고
`<list>`	연결 리스트 (STL)	연결 리스트 구현 참고
`<queue>`	큐 (STL)	큐 구현 참고
`<stack>`	스택 (STL)	스택 구현 참고
`<algorithm>`	정렬, 탐색 (std::sort, std::find)	정렬/탐색 알고리즘 참고

4. 포인터와 메모리 — 자료구조의 핵심

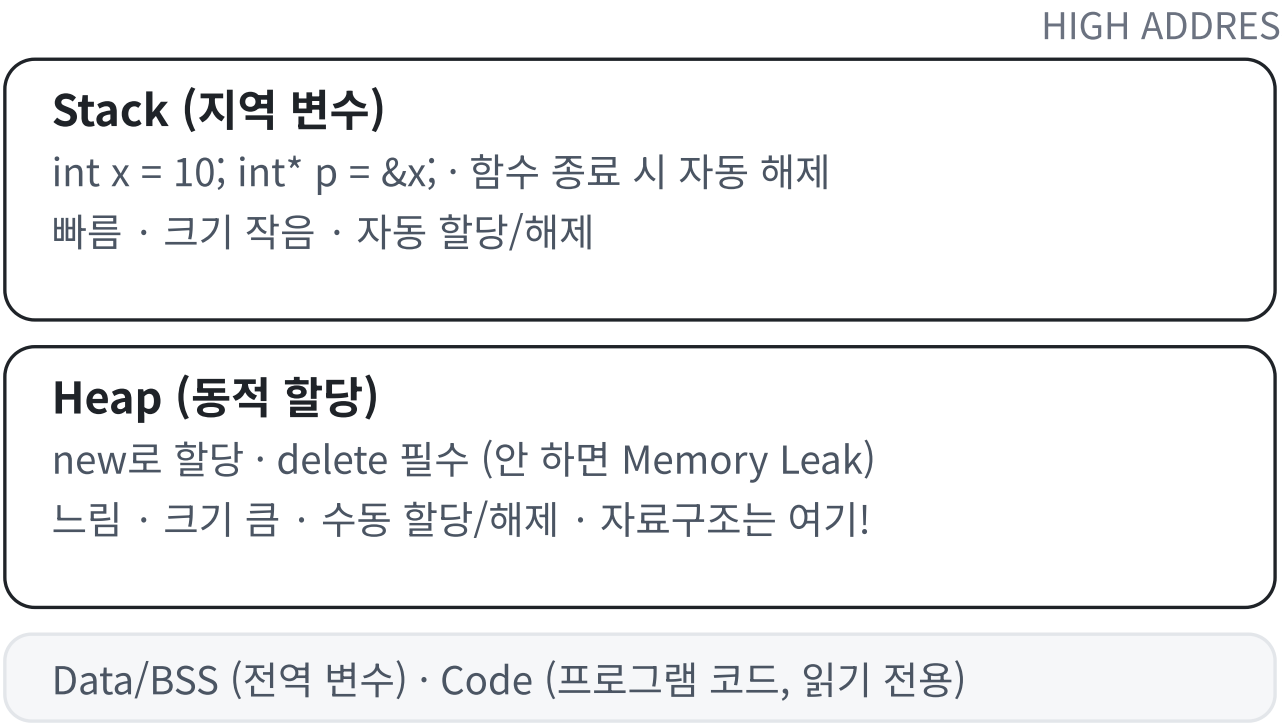
포인터(Pointer): 메모리 주소를 저장하는 변수.

- `&` = "주소를 얻어라" · `*` = "그 주소가 가리키는 객체를 역참조" (읽기·쓰기 모두 가능; 무효 포인터 역참조는 미정의 동작) · `->` = 포인터로 구조체 멤버 접근



포인터의 포인터: pp → p의 주소 → x의 주소 → 값 10

4.1 메모리 구조: Stack vs Heap



이 강의의 동적 자료구조는 new로 Heap에 할당해 만든다

	Stack	Heap
속도	빠름	느림
크기	작음 (한정)	큼
해제	자동	수동 (delete)
위험	자동 해제 (단, 수명 후 접근·댕글링 포인터 가능)	누수·이중 해제·댕글링 위험
용도	지역 변수	동적 배열, **자료구조 필수**

5. 동적 할당: new / delete

크기를 미리 알 수 없을 때 고정 배열은 낭비다. `int arr[1000]` 은 10개만 필요해도 1000개를 잡는다.

```
int n;
cin >> n;                // 사용자가 크기 입력
```

```
int* arr = new int[n];    // n개만 정확히 할당
// ... 사용 ...
delete[] arr;             // 사용 후 반드시 해제
```

메모리 관리는 할당(new) → 사용 → 해제(delete)의 라이프사이클에 대한 책임이다.

Memory Leak(메모리 누수)

```
void processRequest() {
  int* p = new int[1000000];  // 4MB 할당
  // delete[] p;  ← 빼먹으면?
}
```

- 요청 1회당 4MB 낭비 → 3,600회면 4MB × 3600 ≈ 14GB.
- 해제하지 않은 메모리는 프로그램 종료까지 회수되지 않는다 → 장시간 실행 서버는 결국 crash.

자료구조 구현에서는 노드마다 new/delete가 짝을 이뤄야 한다. 메모리 관리 능력 = 자료구조 구현 능력.

6. 주석과 실습

```
int x = 10;  // 단일 라인 주석
/* 여러 줄에 걸친 주석.
  자료구조의 연산 설명, 시간복잡도, delete 주의사항을 적는 데 유용하다. */
```

6.1 실습 1: 포인터와 동적 배열

```
#include <iostream>
using namespace std;

int main() {
  int x = 10, y = 20;

  int* p = &x;            // p가 x를 가리킴
  cout << *p << endl;     // 10
  p = &y;                 // p가 y를 가리키도록 변경
  cout << *p << endl;     // 20

  int n = 5;
  int* arr = new int[n];  // 동적 배열
```

```
    for (int i = 0; i < n; i++) arr[i] = (i + 1) * 100;
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    // 100 200 300 400 500

    delete[] arr;            // 해제 필수!
    return 0;
}
```

6.2 실습 2: 포인터로 만드는 연결 리스트

```
 #include <iostream>
using namespace std;

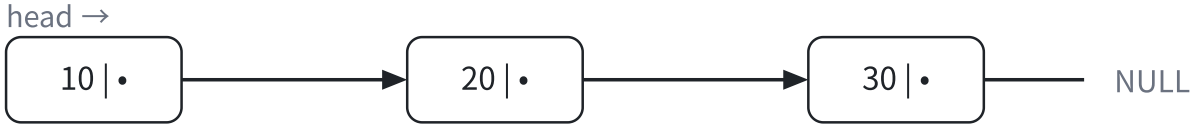
struct Node {
    int data;
    Node* next;              // 다음 노드를 가리키는 포인터
};

int main() {
    Node* head   = new Node();
    Node* second = new Node();
    Node* third  = new Node();

    head->data = 10;  second->data = 20;  third->data = 30;
    head->next = second;  second->next = third;  third->next = NULL;

    Node* current = head;                // 순회
    while (current != NULL) {
        cout << current->data << " → ";
        current = current->next;
    }
    cout << "NULL" << endl;              // 10 → 20 → 30 → NULL

    delete head; delete second; delete third;
    return 0;
}
```



실행 결과의 메모리 구조: 노드 3개가 포인터로 연결

7. 핵심 정리

이 강의에서 C++이 필요한 이유

1. **포인터** — 연결 리스트, 트리, 그래프는 모두 포인터 기반. 포인터가 노드 기반 연결 리스트·트리·그래프 구현의 대표적 방법 (인덱스 기반 구현도 가능).
2. **메모리 관리** — new/delete로 자료구조의 메모리 사용 방식을 이해. 누수는 실무 버그의 주요 원인.
3. **성능 측정** — 빠른 언어라 복잡도 차이를 실측으로 확인하기 좋음 (단, 복잡도 판단은 점근 분석이 중심, 실측은 보조).
4. **개념의 연결** — 저수준(메모리)과 고수준(자료구조)을 잇는 다리.

다음 강의 준비 체크리스트

- 동적 배열 생성: `int* arr = new int[n];`
- 포인터 사용: `Node* next;`
- 구조체 정의: `struct Node { int data; Node* next; };`
- 메모리 해제: `delete[] arr;` / `delete node;`

이 4개가 자연스럽게 나오면 준비 완료.